

Day 2 – .NET Modernization, Quality, Security & Automated Testing

Modernize, Secure, Test and Validate .NET Applications with Claude

.NET

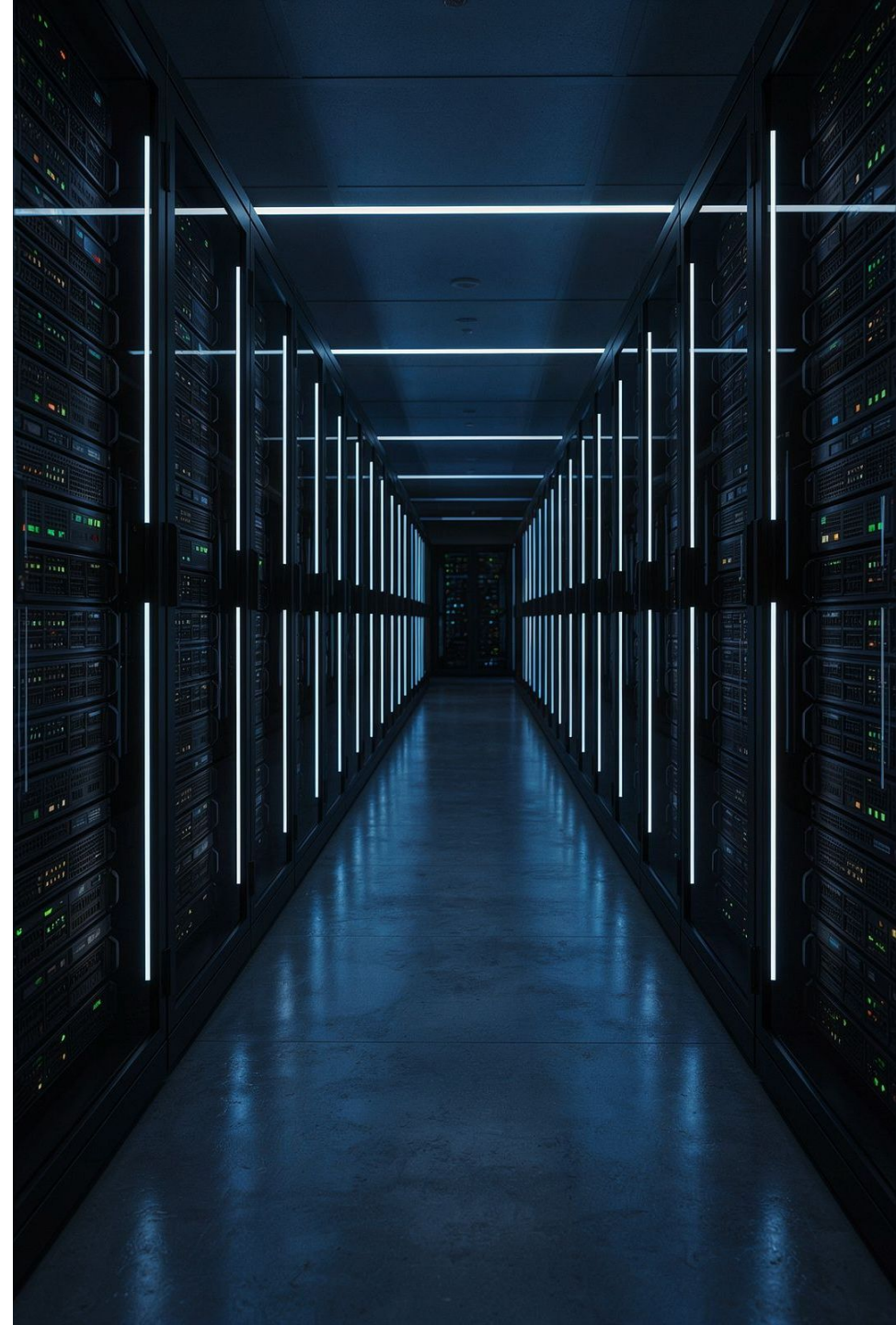
C#

ASP.NET CORE

XUNIT

NUNIT

MOQ



From AI-Generated Code to Production-Quality Code

"Day 1 showed us how to build faster with Claude. Day 2 asks the harder question: how do we make AI-assisted changes safe, correct, and maintainable?"

Speed Without Safety

AI generates code fast, but fast code isn't always correct code

Change Without Tests

Refactoring legacy systems without tests is a recipe for silent regressions

Features Without Security

AI doesn't automatically apply secure coding practices

Day 2 Answers These With:



Legacy Modernization

Analyze, plan, and refactor safely with Claude



Quality & Security

Review AI code, detect bugs, enforce secure patterns



Automated Testing

xUnit, NUnit, mocking, and regression coverage

Day 2 Learning Objectives

1 Analyze Legacy Apps

Understand structure, debt, and modernization opportunities

2 Review & Debug

Detect defects, diagnose root causes, perform security reviews

3 Test & Validate

Generate xUnit/NUnit tests, mock dependencies, build regression suites



Day 2 Agenda

MODULE 5

Legacy .NET Modernization

Analyze → Identify Debt → Plan →
Refactor → Validate

MODULE 6

Code Quality, Bug Detection & Secure Coding

Review → Diagnose → Fix →
Secure → Regression Test

MODULE 7

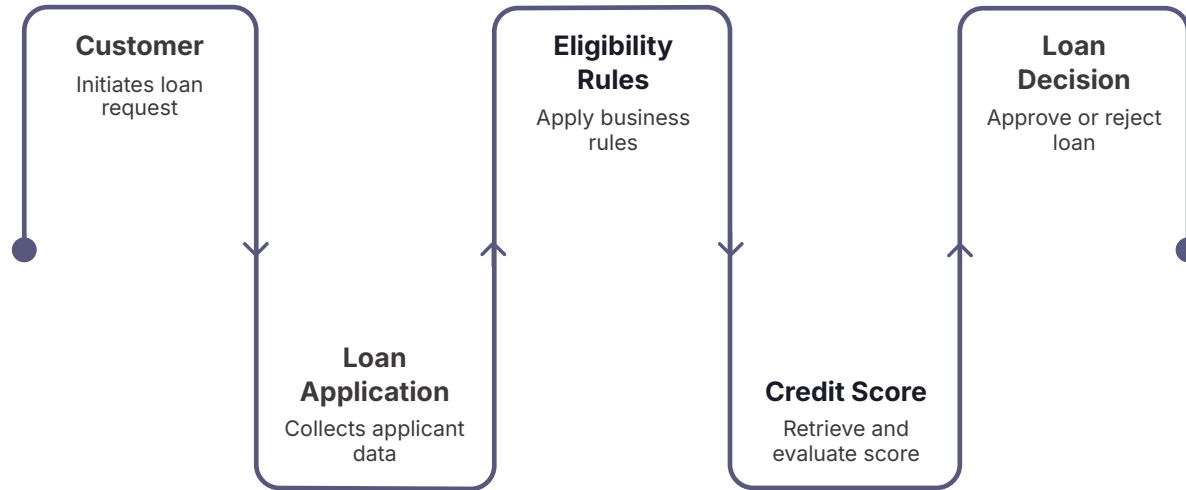
.NET Unit Testing with xUnit & NUnit

Understand → Identify Scenarios
→ Generate Tests → Run →
Improve

- 📄 Automated testing is the safety net for AI-generated code, refactoring, modernization, bug fixes, and security remediation.

Day 2 Case Study: Legacy Loan Processing Application

A simplified enterprise .NET application that processes customer loan requests — used throughout Day 2.



Existing Challenges

Architecture

Business logic inside controllers, tight coupling

Quality

Limited tests, weak validation, poor exception handling

Configuration

Hard-coded values, large methods, no abstractions

Module 5

Legacy .NET Modernization with Claude



Understand



Technical Debt



Modernize



Security



Testability

What is a Legacy .NET Application?

Legacy is about maintainability and risk — not simply age.

A legacy application is one that has become **difficult or risky to change**, regardless of how old it is.

Common Characteristics

Architecture

Monolithic, tightly coupled, static dependencies

Code

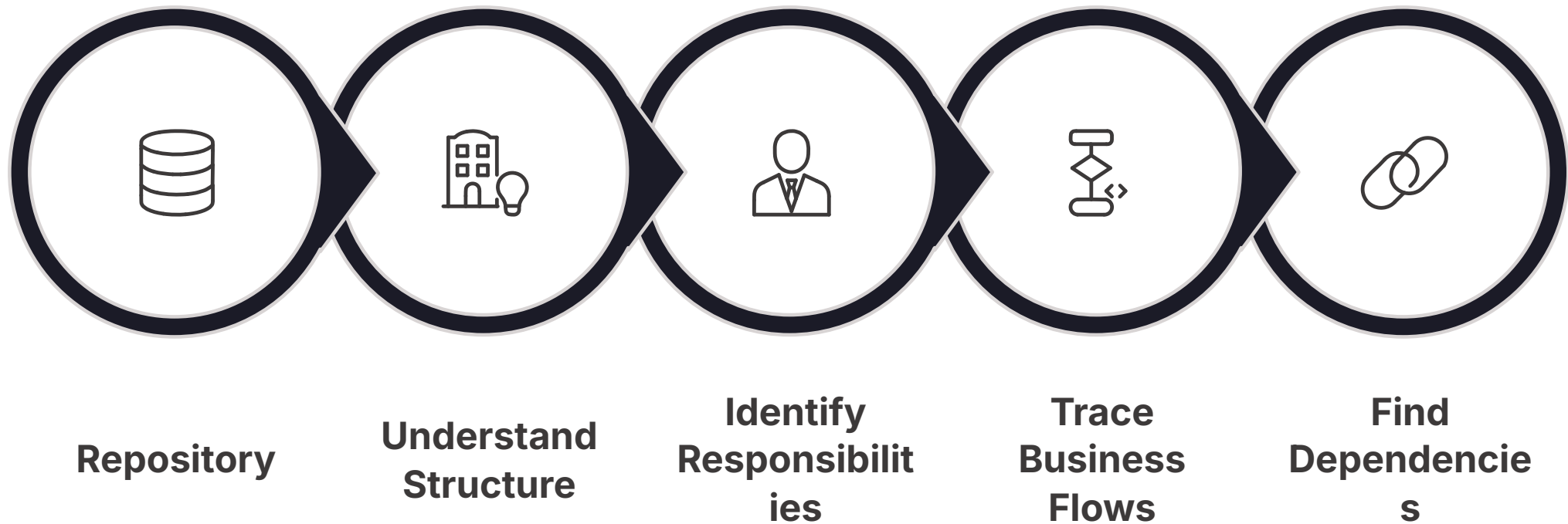
Synchronous APIs, direct DB access, hard-coded config

Quality

Limited tests, outdated libraries, mixed business logic

Understanding a Legacy Application

Before modernizing anything, first understand what the system does — its architecture, dependencies, and risks.



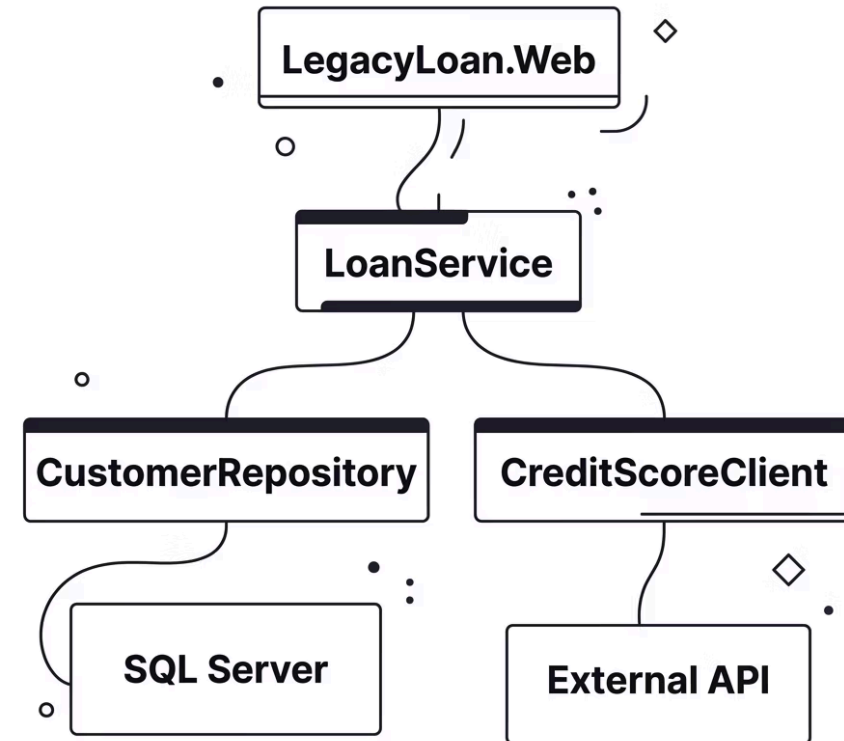
Key areas to examine: architectural layers, business-critical components, external dependencies, database access, authentication, background jobs, and deployment model.

Analyzing Project Structure & Dependencies

Claude Can Identify

- Solution projects and references
- NuGet and shared library dependencies
- Database, external API, and messaging connections
- Static utilities and cross-layer dependencies

📄 Understand coupling before changing architecture.



Identifying Technical Debt

Technical debt is the accumulated cost of decisions that make future changes harder.



Code Issues

Duplicate logic, large methods, god classes, poor naming, dead code



Design Issues

Static dependencies, tight coupling, deep inheritance, missing abstractions



Operational Issues

Missing tests, outdated packages, hard-coded values

Claude can locate these patterns and explain: **Problem → Impact → Risk → Recommended Improvement**

Identifying Outdated Coding Patterns

Common Legacy Patterns

- `.Result` and `.Wait()` on async operations
- Direct SQL inside business services
- Global static state / Service Locator
- Very large controllers
- `new` dependencies inside business logic
- Generic `catch(Exception)`
- Hard-coded connection strings

Important

Not every old pattern needs immediate replacement.

Modernization should be driven by:

Business Value + Risk + Maintainability

Modernization Opportunities



Code Level

Readability, reduce duplication, introduce async, improve error handling



Architecture Level

Separate responsibilities, introduce DI, create domain boundaries



Platform Level

Upgrade runtime, upgrade libraries, improve configuration management



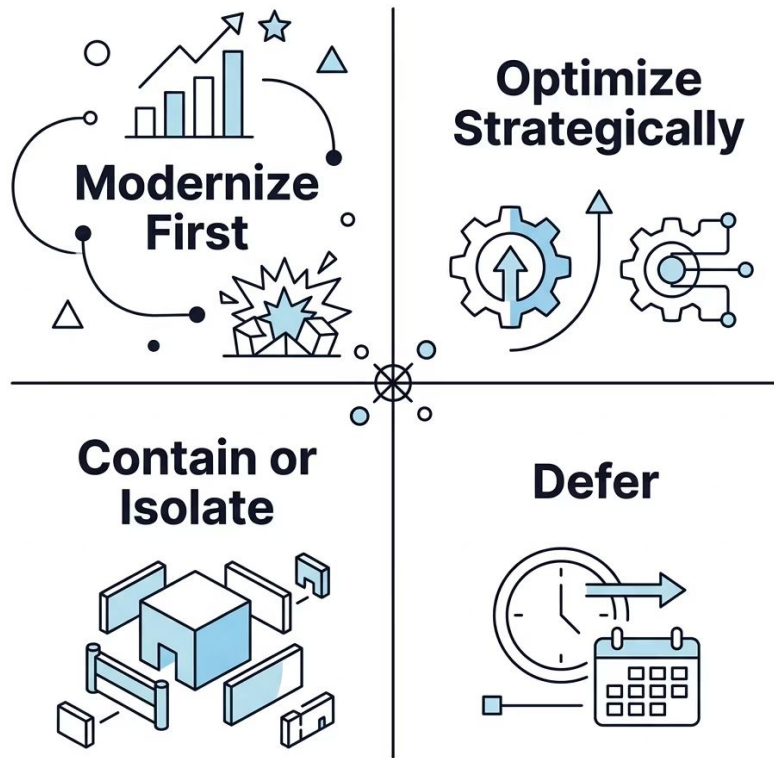
Quality Level

Add tests, static analysis, improve observability

📌 Modernization does not automatically mean microservices.

Modernization Prioritization

Not all technical debt has equal priority. Use a risk-driven model to decide what to tackle first.



Modernization should be incremental and risk-driven.

AI-Assisted Modernization Plan

Ask Claude to produce a **modernization plan** rather than immediately rewrite code.

01

Current Architecture

Document existing structure and responsibilities

03

Target State

Recommended architecture and migration phases

02

Technical Debt & Risks

Security, testability, and performance concerns

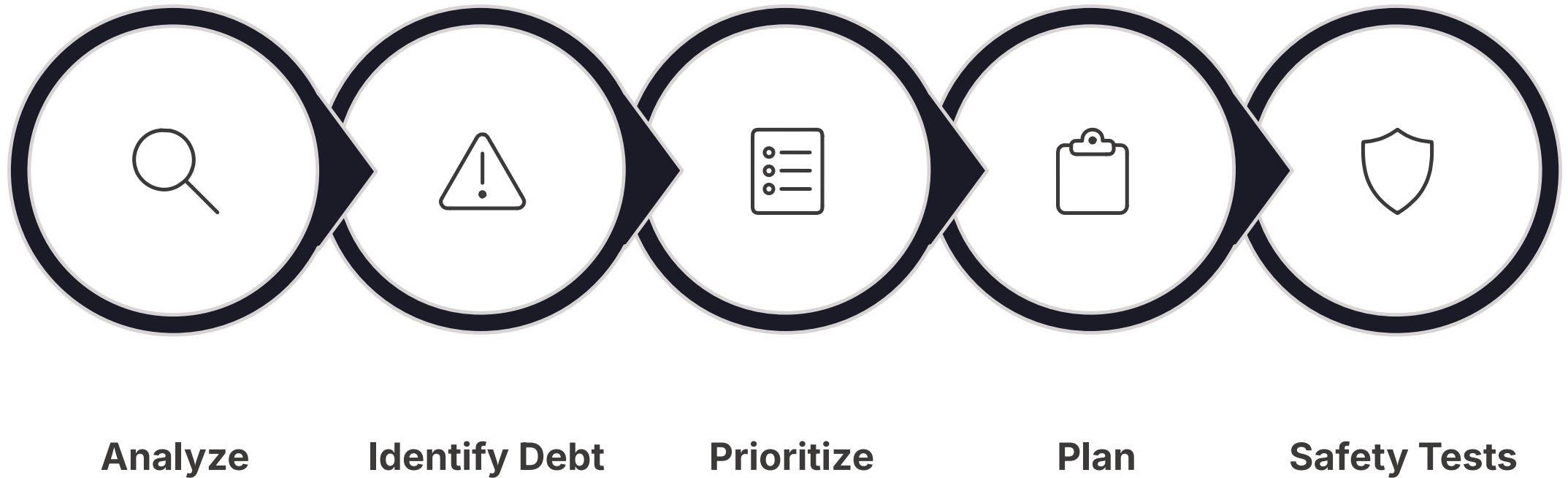
04

Validation Plan

Required tests, risk level, rollback considerations

Analyze first. Refactor second.

Legacy Modernization Workflow



❏ Modernization should preserve existing business behaviour unless change is explicitly required.

Refactoring Legacy C# Code

Claude Can Assist With

- Method extraction and dependency isolation
- Removing duplication, introducing interfaces
- Separating business rules from infrastructure
- Replacing static dependencies
- Async modernization and control flow simplification

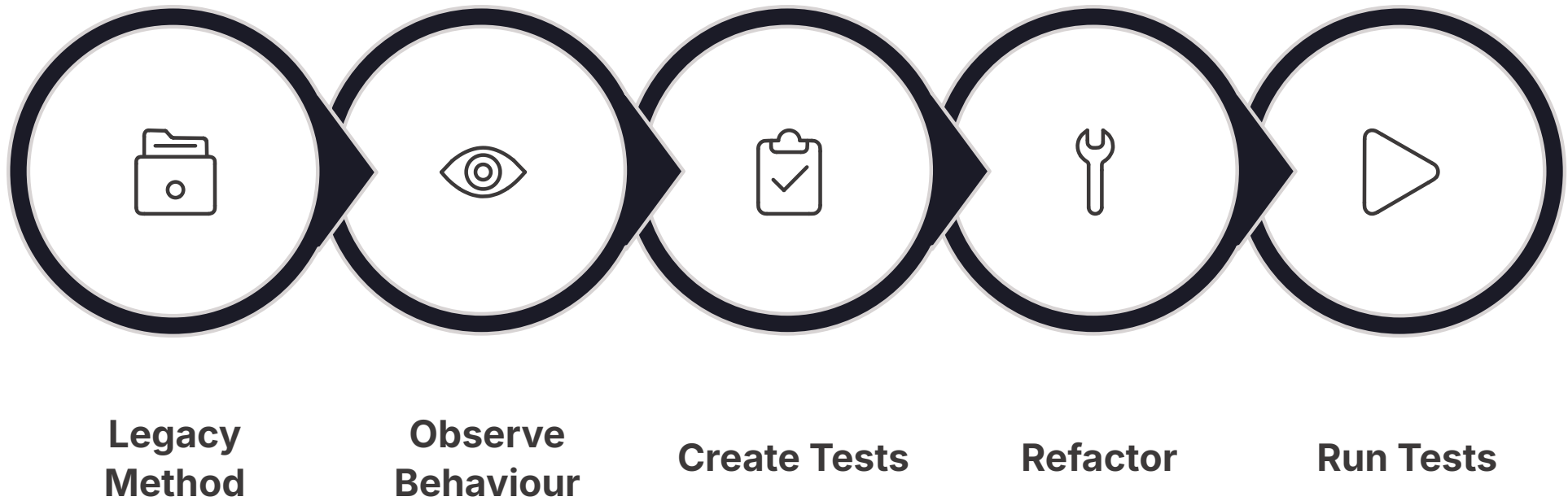
Refactoring Rule

Before changing any code, ask:

What behaviour must remain unchanged?

Characterization Tests Before Refactoring

Legacy code may have little documentation. Capture existing behaviour first — before changing anything.



Before improving legacy code, first make its behaviour measurable.

Improving Maintainability

Before

Controller → Database → Business Rule → External API — all inside one method

After

Controller → Application Service → Domain Logic → Repositories / External Services

Modernized Code Should Improve

- Readability and separation of concerns
- Dependency management and testability
- Error handling and naming
- Reusability and change isolation

Modernization – Security, Testability & Performance



Security

- Hard-coded secrets
- Weak validation
- Outdated dependencies
- Unsafe SQL, sensitive logs



Testability

- Static dependencies
- Tight coupling
- Hidden side effects
- Non-deterministic behaviour



Performance

- Blocking calls
- N+1 queries
- Large data loads
- Repeated remote calls

Validating AI-Generated Modernization



❏ A cleaner implementation is not useful if it changes critical business behaviour.

Lab 1

Legacy .NET Modernization with Claude

Analyze a legacy .NET application, identify technical debt, create a modernization plan, refactor a selected component and validate the generated changes.



Module 6

Code Quality, Bug Detection & Secure Coding

Code Quality

Improve maintainability

Bug Detection

Improve correctness

Secure Coding

Reduce security risk



AI-Assisted Code Review

Claude Reviews For

- Correctness, readability, maintainability
- Complexity and duplication
- Architecture and error handling
- Testability, security, performance

📌 AI review complements — not replaces — human review.

Recommended Output Format

Severity	Finding	Impact	Recommendation
High	Example finding	Example impact	Example fix

Code Smells

Code smells are indicators of potential design problems — not automatically bugs, but signals that code deserves closer inspection.



Structural Smells

Long Method, Large Class, Too Many Parameters, Deep Nesting



Logic Smells

Duplicate Code, Long Conditional Chains, Primitive Obsession



Coupling Smells

Feature Envy, Tight Coupling, Global State

Complexity & Duplication Detection

Complexity

Deep nesting, multiple responsibilities, large switch statements, and many exit paths make code harder to understand, test, and maintain.

Duplication

Repeated business rules cause inconsistent behaviour when only one copy is updated.

Refactoring Goal: Reduce cognitive load while preserving behaviour.

Reviewing AI-Generated Code

Correctness

Does it compile? Does it solve the requested problem?

Architecture

Does it follow architecture? Did it introduce unnecessary dependencies?

Safety

Is error handling correct? Is it secure and testable?

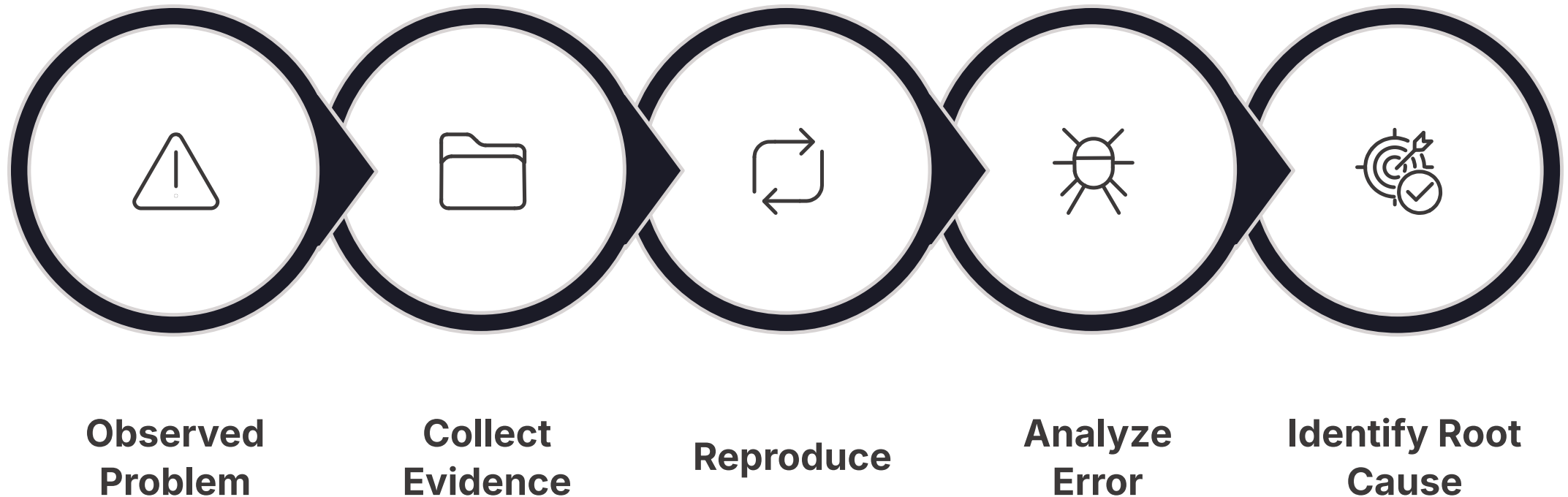
Compatibility

Did it modify unrelated code? Does it preserve backward compatibility?

Would we approve this code if a human developer submitted it?

Bug Detection Workflow

A production defect should not immediately trigger random code changes. Follow a structured diagnostic process.



Analyzing Errors & Stack Traces

A Stack Trace Reveals

- Exception type and failure location
- Call sequence and framework components



A `NullPointerException` does not automatically mean "add a null check."

Ask: **Why was the value null?**

Possible Root Causes

Invalid input or missing database record

Incorrect mapping or failed dependency

Incorrect state transition

Fix root causes, not symptoms.

Root-Cause Analysis with Claude

01

Observed Behaviour

What happened?

03

Evidence

Logs, stack trace, request, environment

02

Expected Behaviour

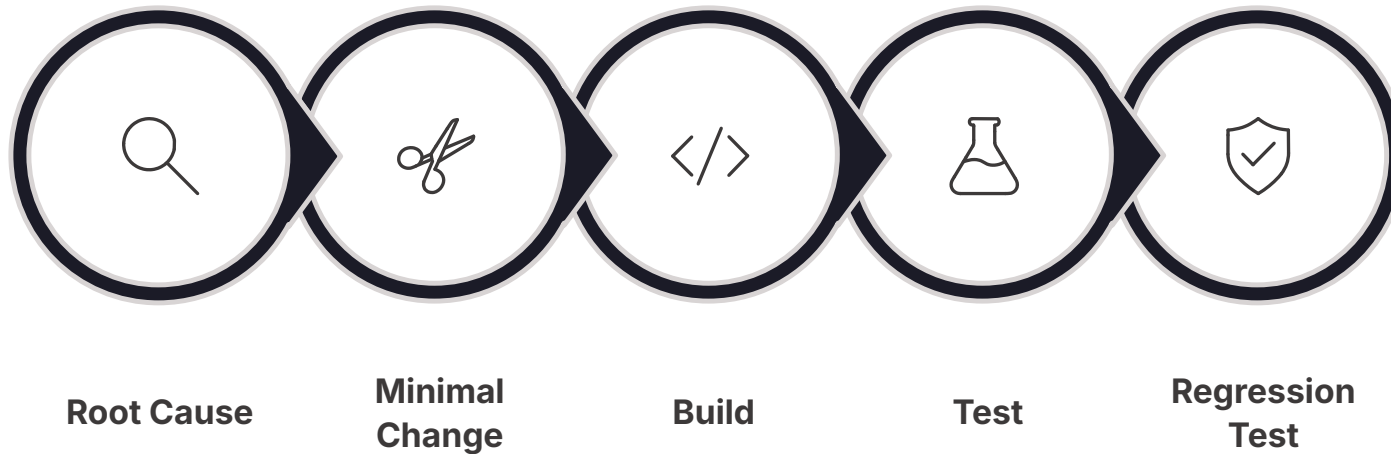
What should happen?

04

Claude Output

Hypotheses → Diagnostic steps → Most probable root cause → Minimal remediation

Generating & Validating Fixes

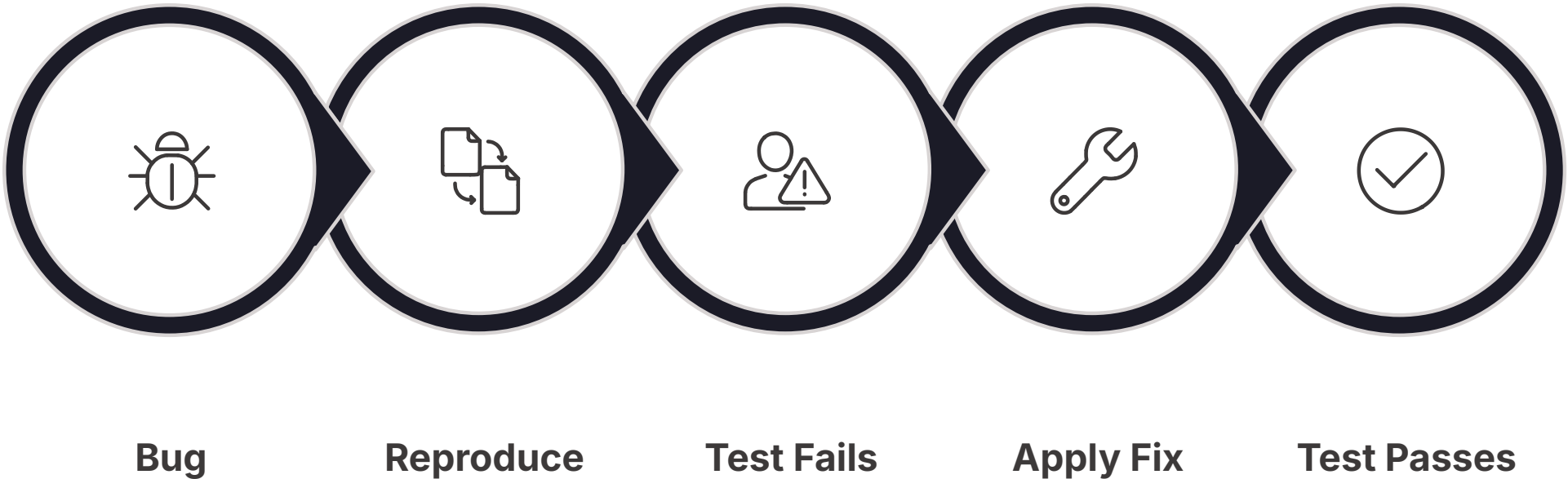


Smallest safe change wins.

⚠ Avoid allowing AI to rewrite large areas of the application for a small defect.

Regression Testing

A bug fix should include a test that proves the defect remains fixed — permanently.



- ✅ Future developers cannot accidentally reintroduce the same defect without detection.

Secure Coding with Claude

Security Review Areas

- Input validation and authentication
- Authorization and SQL injection
- Hard-coded secrets and sensitive data
- Error handling, logging, and API exposure
- Dependency security

☐ Security review should be part of normal code review.



Input Validation & Injection Risks

Risks

- SQL injection and command injection
- Path traversal and invalid state
- Resource abuse

Safer Practices

- Validate all inputs
- Use parameterized queries
- Use ORM correctly
- Enforce domain constraints
- Reject unexpected values

Security Question: Where does this data come from, and where does it go?

Hard-Coded Secrets

Never Store in Source Code

- API keys and passwords
- Database credentials and tokens
- Certificates

Better Approach

Application → Configuration → Secret Management

- Azure Key Vault
- AWS Secrets Manager
- Environment-based configuration



Authentication vs Authorization

Authentication — Who are you?

JWT, Entra ID, Identity Provider

Authorization — What are you allowed to do?

Roles, Claims, Policies, Resource ownership

 Common problem: Endpoint verifies authentication but does not check whether the user is allowed to access the requested resource.

Sensitive Data Exposure

Avoid Exposing

- Passwords, tokens, financial data
- Personal data and internal stack traces
- Internal system identifiers where unnecessary

Review These Areas

- API responses, logs, exceptions
- Telemetry and database queries

Collect and expose only what is required.

Secure Error Handling & Logging

Poor Approach

Return the complete exception and stack trace to users.

User Receives

Safe error message, appropriate HTTP status, correlation/reference ID

Internal Logs Receive

Technical detail, context, correlation ID — but never passwords, tokens, or sensitive customer data

Security Review of AI-Generated Code

Ask Claude to Inspect

- Trust boundaries and user inputs
- Authentication and authorization
- Secret handling and external calls
- Database operations, logging, error handling

Expected Output

Severity → Vulnerability → Exploit Scenario → Impact
→ Recommended Fix

❏ Security recommendations must still be independently validated.

Lab 2

Code Quality, Bug Remediation & Secure Coding

Analyze code, identify a defect, determine the root cause, implement a fix, perform a security review, create a regression test and validate the result.





Module 7

.NET Unit Testing with xUnit & NUnit

Automated tests provide the safety net required for AI-generated code, refactoring, modernization, bug fixes, security remediation, and continuous delivery.

Why Automated Testing Matters



Verify & Protect

Verify business rules, detect regressions,
reproduce defects

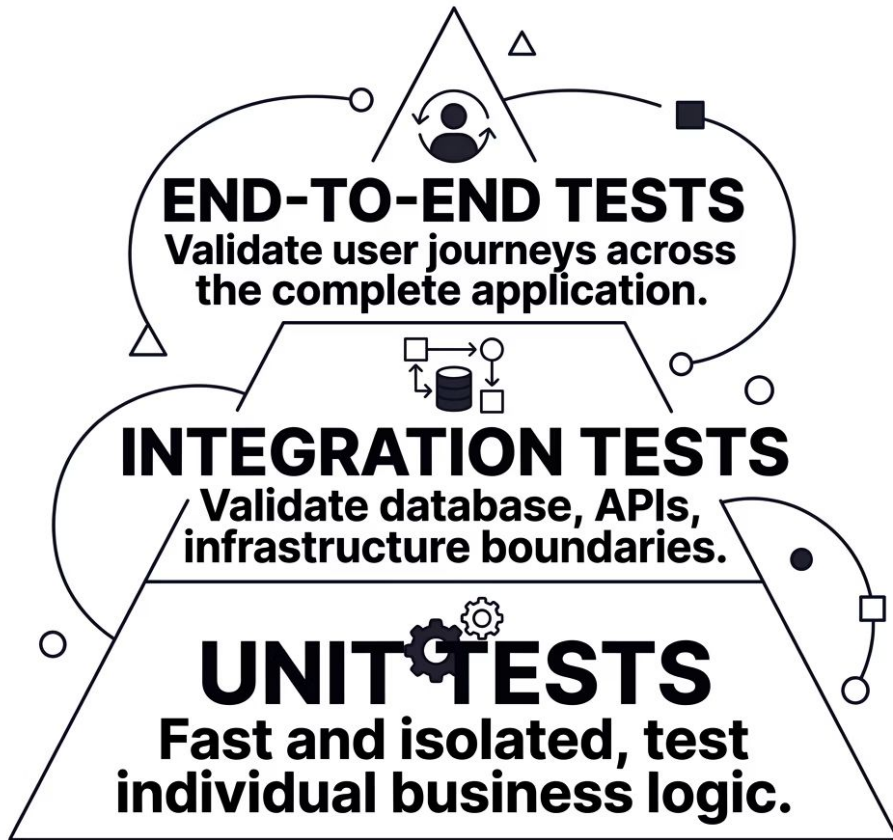
**AI makes code generation faster. Testing must make
validation equally fast.**



Enable Speed

Validate AI-generated code, refactor confidently,
support CI/CD quality gates

Testing Pyramid – Practical View



Unit Tests

Fast and isolated. Test individual business logic.

Integration Tests

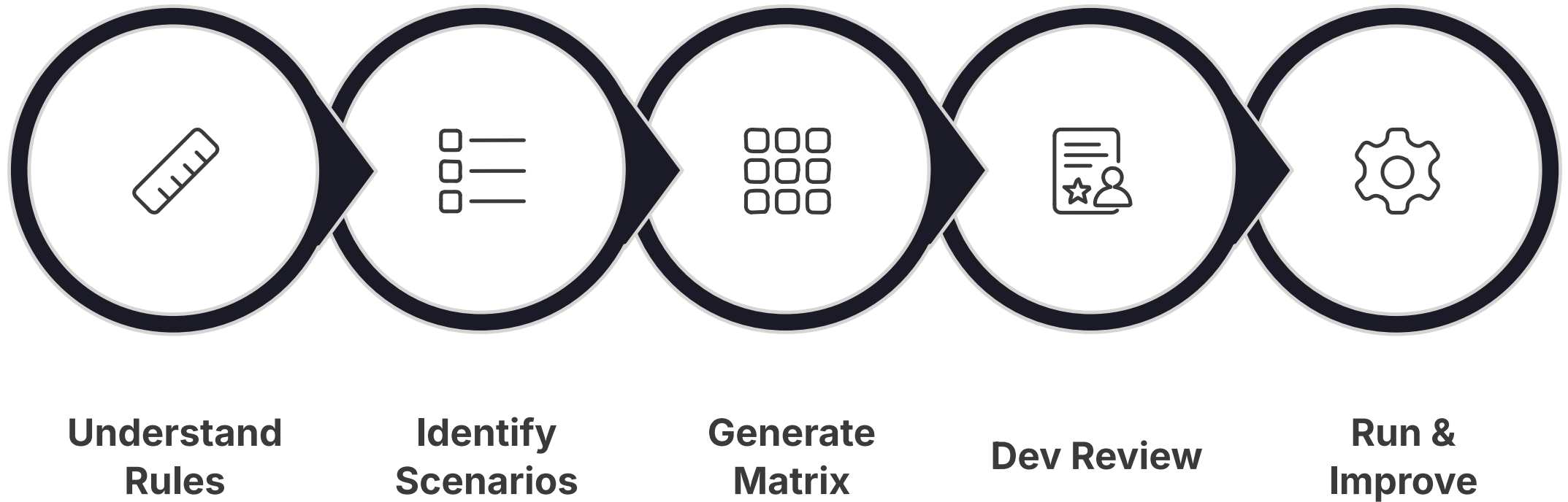
Validate boundaries: database, APIs, infrastructure.

End-to-End Tests

Validate user journeys across the complete application.

❏ Use the right level of testing for the right risk.

AI-Assisted Testing Workflow



⚠ Do not start by asking Claude only: "Generate unit tests."

Test Matrix Before Test Code

Example Business Rule

Loan applications below a minimum credit score are rejected.

Why Test Matrices Matter

- Identify missing requirements
- Surface boundary conditions
- Expose negative cases and dependency failures

Test Scenarios

Score above threshold

Score exactly at
threshold

Score below threshold

Missing or invalid
score

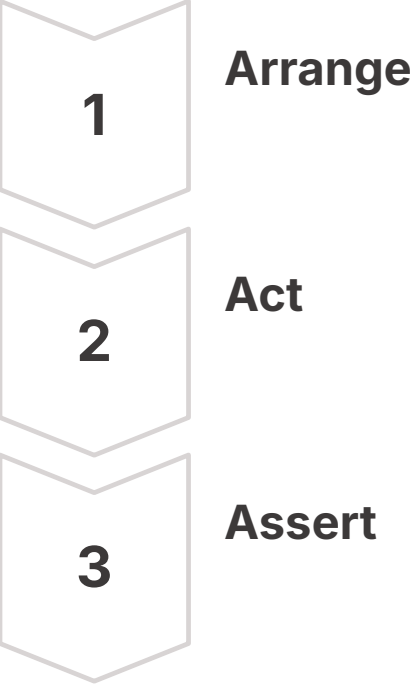
External credit service failure

xUnit Fundamentals

Core Concepts

Attribute	Purpose
[Fact]	Single test scenario
[Theory]	Parameterized test
[InlineData]	Provide parameter values

Typical Pattern



Also covers: exception testing, async tests, fixtures, and test naming.

xUnit – Fact Example

```
[Fact]
public void CalculateInterest_ShouldReturnExpectedValue()
{
    // Arrange
    var service = new LoanCalculator();
    // Act
    var result = service.CalculateInterest(10000, 10);
    // Assert
    Assert.Equal(1000, result);
}
```

Arrange

Prepare data and dependencies

Act

Execute the behaviour

Assert

Verify the result

xUnit – Theory & InlineData

Parameterized tests allow multiple scenarios using one test method — less duplicate test code.

```
[Theory]
[InlineData(1000, 10, 100)]
[InlineData(2000, 10, 200)]
[InlineData(5000, 5, 250)]
public void CalculateInterest_ShouldReturnExpectedValue(
    decimal amount, decimal rate, decimal expected)
{
    // Test
}
```


Positive, Negative & Boundary Testing



Positive

Valid loan application is accepted



Negative

Invalid customer is rejected



Boundary

Credit score exactly equals minimum limit



Error

Credit service throws an exception

Test behaviour around the edges.

Async Unit Testing

Modern .NET applications frequently use asynchronous APIs. Test async behaviour using async test methods.

```
[Fact]
public async Task GetLoan_ShouldReturnLoan_WhenItExists()
{
    var result = await service.GetLoanAsync(id);
    Assert.NotNull(result);
}
```

⚠ Avoid `.Result` and `.Wait()` inside asynchronous tests.

NUnit Fundamentals

Core Concepts

Attribute	Purpose
[Test]	Single test
[TestCase]	Parameterized test
[SetUp]	Runs before each test
[TearDown]	Runs after each test

NUnit is a mature and widely adopted framework. [SetUp] and [TearDown] serve as per-test lifecycle hooks, including support for asynchronous setup/teardown methods.

Additional areas: assertions, async testing, fixtures, and setup/cleanup lifecycle.

NUnit Example

```
[Test]
public void CalculateInterest_ShouldReturnExpectedValue()
{
    var service = new LoanCalculator();
    var result = service.CalculateInterest(10000, 10);
    Assert.That(result, Is.EqualTo(1000));
}
```

```
[TestCase(1000, 10, 100)]
```

```
[TestCase(2000, 10, 200)]
```

```
public void CalculateInterest_ShouldReturnExpectedValue(
    decimal amount, decimal rate, decimal expected)
{
    // Test logic
}
```

xUnit vs NUnit

Area	xUnit	NUnit
Basic Test	[Fact]	[Test]
Parameterized	[Theory] + [InlineData]	[TestCase]
Per-Test Setup	Constructor / fixtures	[SetUp]
Cleanup	IDisposable / fixtures	[TearDown]
Async Tests	Supported	Supported
Positioning	Common in modern .NET	Mature, widely adopted

📌 Use the framework that fits the existing project, team and client standards.

Test Naming Conventions

Pattern 1: Method_Scenario_ExpectedResult

CreateLoan_WhenCreditScoreTooLow_ShouldRejectApplication

Pattern 2: Given_When_Then

GivenInvalidCustomer_WhenCreatingLoan_ThenApplicationIsRejected

Good Test Names Help

Understand failures quickly

Document business rules

Maintain the test suite over time

Assertions & Exception Testing

Assertions May Validate

- Return value and state change
- Exception thrown
- Collection contents
- Dependency interaction

Exception Scenario Example

"Loan service should throw
`CustomerNotFoundException` when customer does not
exist."

❏ Avoid asserting irrelevant implementation details.

Using Claude to Generate Better Unit Tests

Bad Prompt

"Generate tests."

Better Prompt Structure

01
Role
Senior .NET test engineer
02
Context
LoanService processes loan applications
03
Business Rules
Min credit score = 650, amount must be positive, customer must exist
04
Task
Generate test scenarios — no code yet. Output: matrix grouped by positive, negative, boundary, and dependency failure cases

Recommended AI Testing Workflow



□ The developer validates whether AI identified the correct business scenarios.

Mocking Dependencies

LoanService Depends On

- CustomerRepository
- CreditScoreService
- NotificationService

In a Unit Test

Replace all three with mocks so LoanService is tested in isolation.

Benefits of Mocking

Fast

No real I/O or network calls

Predictable

Controlled dependency behaviour

Isolated

Tests only the unit under test

Why Mocking is Required

Mocking allows developers to control dependency behaviour and verify how **LoanService** responds — without calling the real Credit Score API.

Scenario 1

Credit service returns score: 720

Scenario 2

Credit service returns score: 590

Scenario 3

Credit service throws timeout exception

Mocking Success & Failure Paths

Dependency Success

- Repository returns customer
- Credit service returns score

Dependency Failure

- Customer does not exist
- Credit service fails
- Notification service throws error

Testing only successful dependency calls creates false confidence.

Avoid Excessive Mocking

Bad Test

Verifies every private/internal step — tightly coupled to implementation.

Better Test

Verifies externally observable behaviour.

Mock These

- Databases and external APIs
- File systems and messaging
- Infrastructure boundaries

Don't Mock

Simple domain objects or pure business logic unnecessarily.

Verifying Interactions

Example

An approved loan should trigger `NotificationService.SendApprovalAsync()`. A test may verify this call was made.

Rule

Only verify interactions that are part of the **required behaviour**.

Do not verify every internal call.

Running Tests with dotnet test

```
dotnet test
```



Newer .NET SDKs support both traditional VSTest-based and Microsoft.Testing.Platform-based execution depending on project configuration. A failed test command can prevent pull request merge, deployment, or release promotion.

Analyzing Failing Tests with Claude

Provide Claude

- Test name, expected result, actual result
- Stack trace and relevant production code
- Business requirement

Ask Claude: Is the Problem...

Production code defect?

Incorrect test expectation?

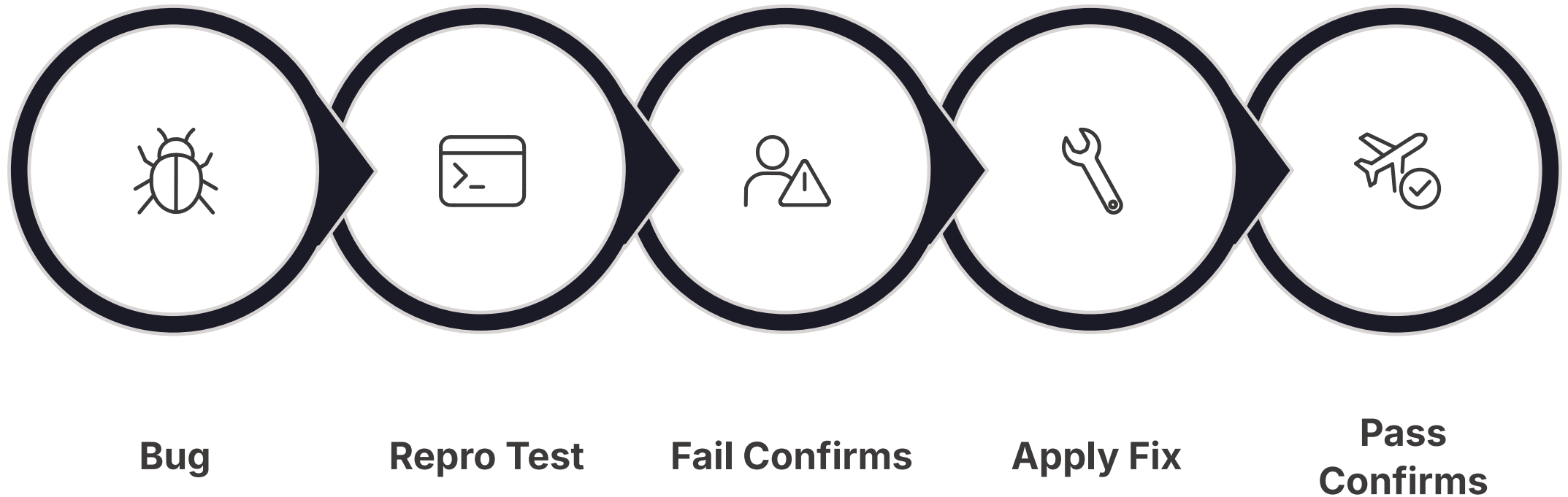
Mock configuration issue?

Ambiguous requirement?

Environmental issue?

Diagnose first. Fix second.

Regression Testing



- ✓ The defect becomes much harder to accidentally reintroduce.



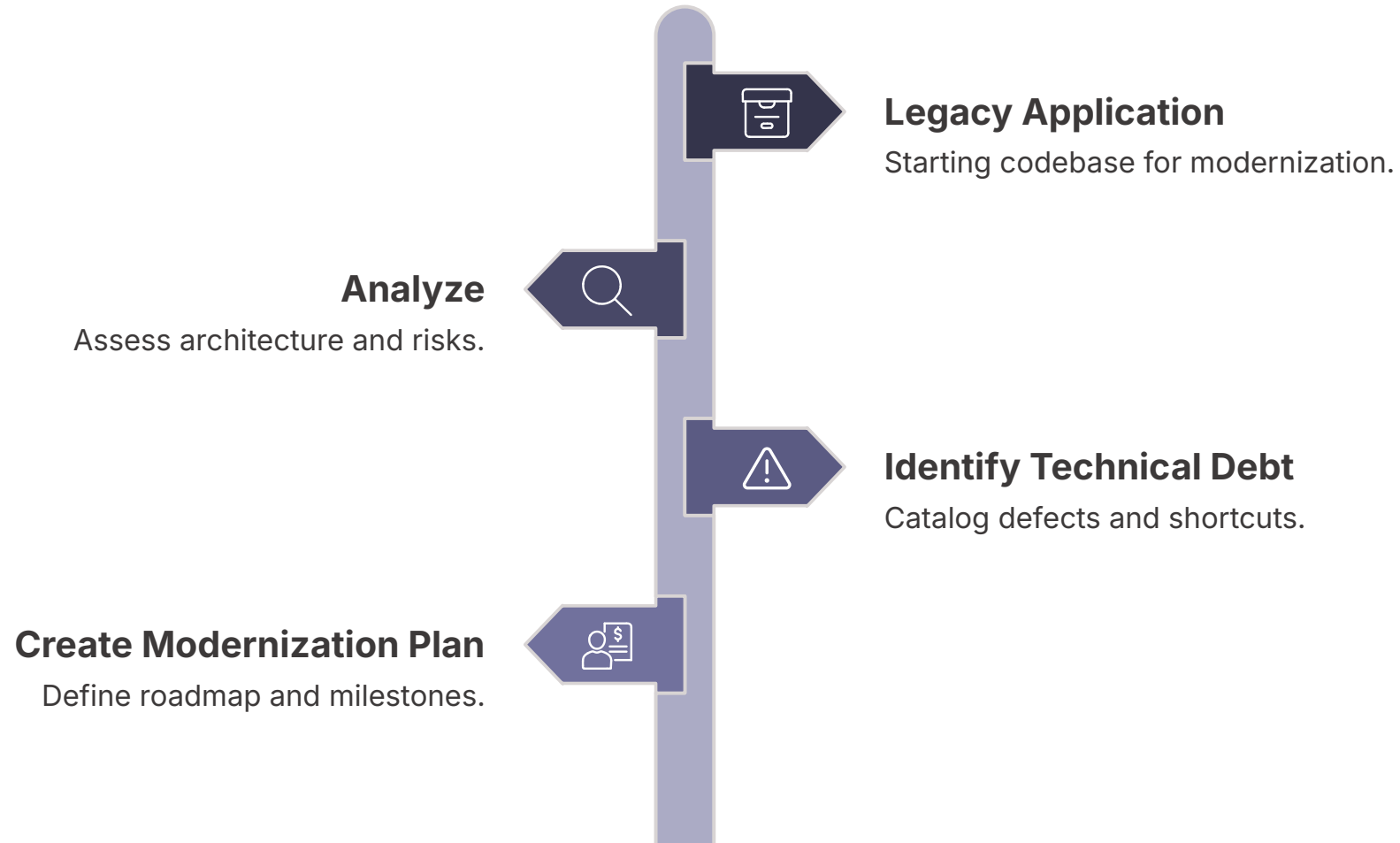
Lab 3

AI-Assisted Unit Testing with xUnit & NUnit

Use Claude to analyze a .NET service, create a test matrix, generate unit tests, mock dependencies, execute tests, diagnose failures and compare xUnit with NUnit conventions.

Day 2 Integrated Quality Workflow

From Legacy Code to Validated Modern Code



AI + Testing = Safer Modernization

✗ Without Tests

AI Change → Developer Review → *Hope behaviour remains correct*

✓ With Tests

AI Change → Build → Unit Tests → Regression Tests
→ Security Checks → Review → **Evidence-Based Confidence**

Automated tests convert assumptions into executable evidence.

Day 2 Key Takeaways



Modernize Incrementally

Understand legacy behaviour before changing it; use characterization tests as your safety net.



Review Everything Claude Generates

AI code is a starting point, not a finished product; always review for correctness and security.



Diagnose Before You Fix

Root-cause analysis prevents the same bug from recurring in a different form.



Security is a First-Class Concern

Validate inputs, protect secrets, and enforce authorization on every endpoint.



Tests Are Not Optional

Automated tests are what allow AI-assisted development to move fast without breaking things.

**AI Can Accelerate Change — Tests
Make That Change Safe**

**Move fast — but
never without a
safety net.**

 Claude Accelerates

Modernization, refactoring, bug fixes, and test generation happen in minutes, not days.

 Tests Protect

Every AI-assisted change is validated by automated tests that run on every build.

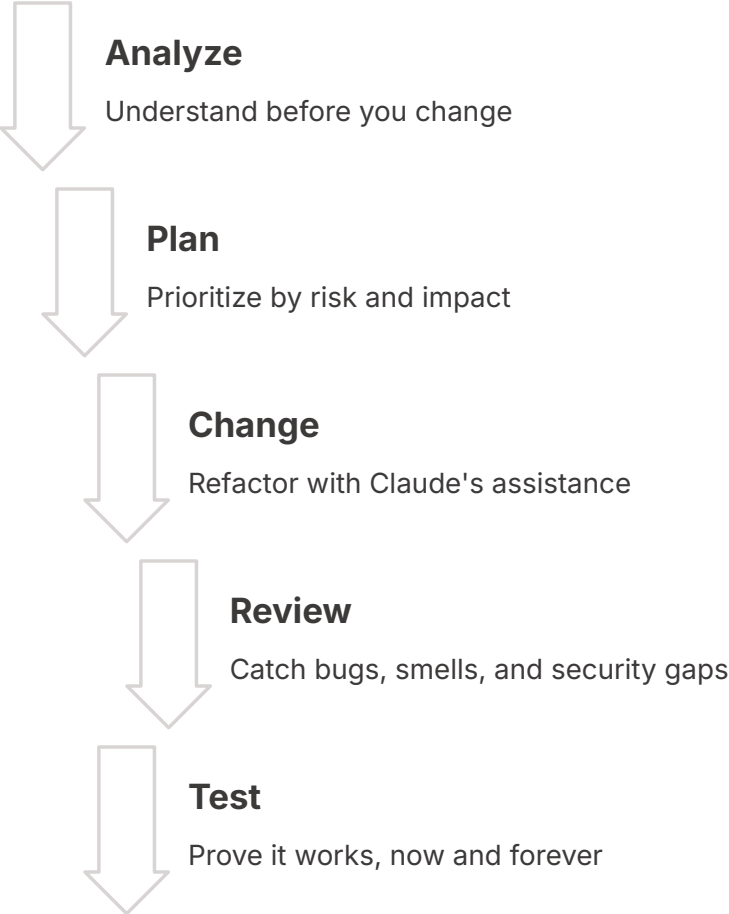
 You Stay in Control

Review every suggestion. Understand every change. Own every decision.



Final Mental Model

AI accelerates the work. Tests and reviews make that work trustworthy.



AI is a Tool, Not an Oracle

Always review, validate, and test AI-generated code



Security is Non-Negotiable

Every change must be reviewed for injection, secrets, and auth



Tests are the Safety Net

No refactor or fix is complete without automated test coverage